Software for Autonomous Driving using Supervised & Reinforcement Learning

Deployed on a 4WD chassis equipped with HC-SR04 ultrasonic sensors and a 5MP camera, powered by Raspberry Pi 4.

Chiranth D Nandi, 25th March 2026.

MobNetV3 Small Classifier → Behavioural Cloning → PPO + Transformer, LSTM → INT8 ONNX Deployment

Tech Stack: Python 3.17, PyTorch, Stable-Baselines3 (PPO, RecurrentPPO), Transformer, LSTM, MobileNetV3, ONNX Runtime, INT8 Quantization, Gymnasium, Raspberry Pi.

1. Project Overview

This project implements a complete end-to-end autonomous driving pipeline for a 4WD robotic car, designed and built by B.Tech AIML students for an ARM Based Project. The system navigates a three-lane road, avoids stationary obstacles, waits for oncoming traffic to clear, and executes safe lane-change manoeuvres.

1.1 Hardware

Component	Specification
Chassis	4WD car chassis (academic scale)
Compute	Raspberry Pi 4B (2 GB RAM, quad-core Cortex-A72)
Motor Drivers	2× TB6612FNG dual H-bridge (3× AA cell supply)
Camera	Raspberry Pi Camera Module (5 MP)
Range Sensors	3× HC-SR04 ultrasonic (left −35°, centre 0°, right +35°)
Power	10000 mAh USB power bank → Pi; 3× AA → motors (separate rails)
LCD Display	240x320 2.4inch LCD Display

1.2 Core Scenario

Three lanes with Traffic Signals and signs at periodic distances and obstacles. The agent must

- Detect the stationary obstacle (if any) in its current lane via ultrasonic sensor (centre reading drops toward 0).
- Detect the oncoming vehicle via the ObstacleTracker velocity estimator coupled with the Left side ultrasonic sensor aligned at 30 degrees yaw to left.
- Decide the best lane option to switch lane (right or left).
- Wait until the oncoming vehicle (if any) has cleared, using learned temporal reasoning (LSTM).
- Execute a lane-change, pass the obstacle, and return to centre.

Why this is non-trivial
A naive rule-based system would either collide (no temporal awareness) or wait forever (no learned urgency). The **Transformer gives cross-sensor attention**; the **LSTM gives temporal memory of the oncoming vehicle's trajectory**; PPO trains the policy to optimise long-horizon reward rather than greedy per-step survival.

2. System Architecture

2.1 Pipeline Overview

The pipeline has three sequential stages that run in order during training and are then frozen at deployment:

Stage	Purpose & Output
Stage 0 — MobNetV3 Classifier	Fine-tuned MobileNetV3-Small classifier for traffic sign and signal recognition. Trained on labelled images of red lights, green lights, and stop signs (**96×96 input**). Output: best_classifier.pth, exported to classifier_int8.onnx for on-device inference.
Stage 1 — Behavioural Cloning (BC)	Supervised pre-training of the Transformer feature extractor on expert demonstrations. Output: a warm-started weight file best_bc.pth.
Stage 2 — Proximal Policy Optimisation (PPO)	RL fine-tuning of the full Transformer + LSTM + policy head. BC weights are loaded into the feature extractor; the LSTM and heads are trained from scratch. Output: final_model.zip.
Stage 3 — ONNX Export + INT8 Quantisation	The trained PyTorch policy is exported to ONNX FP32, then quantised to INT8 for Raspberry Pi deployment. Output: policy_int8.onnx, classifier_int8.onnx. **Reduced memory usage by 67-68% for better edge deployment.**

2.2 Neural Network Architecture

The policy network is a Transformer → LSTM → MLP stack:

```
Input (15,) → Embedding (15, 32) → Positional Encoding → 2× TransformerBlock → Flatten
(480,) → MLP → (128,) → LSTM(64) → Policy Head (5,) / Value Head (1,)
```

Module	Shape / Detail
feature_embedding	nn.Linear(1, 32): each scalar sensor value → 32-dim token
pos_encoding	nn.Parameter(1, 15, 32): learnable positional IDs
TransformerBlock × 2	MultiHeadSelfAttention(d=32, h=4) + FFN(32→128→32) + LayerNorm + Residual
output_proj	nn.Linear(480, 256) → GELU → Dropout → nn.Linear(256, 128)
lstm_actor	nn.LSTM(128, 64, num_layers=1, batch_first=False)
action_net (policy head)	nn.Linear(64, 64) → ReLU → nn.Linear(64, 5) → Softmax
value_net (critic head)	nn.Linear(64, 64) → ReLU → nn.Linear(64, 1)
Total parameters	≈ 137 000 (FP32 ≈ 0.52 MB; INT8 ≈ 0.13 MB)

3. Architectural Decisions — Why This, Not That

3.1 Why a Transformer Feature Extractor?

The 15-dimensional state vector contains sensors that are not independent. Consider:

- us_center = 0.2 (obstacle very close ahead)
- oncoming_det = 1.0 (oncoming vehicle exists)
- left_free = 1.0 (left lane appears clear)

A plain MLP applies the same weight matrix regardless of context. It cannot learn 'left_free matters more when an obstacle is imminent and less when the road is clear.' Self-attention is precisely the mechanism that computes context-dependent feature weighting.

Why not a CNN on the raw camera frame? The Raspberry Pi 4B delivers ≈ 15 FPS with YOLOv8-nano at 320×240. Running a CNN policy on raw pixels in addition would saturate the CPU. Our approach separates perception (YOLO + ultrasonic → 15-float state) from decision-making (Transformer + LSTM), keeping inference at < 5 ms per forward pass.

3.2 Why an LSTM on top of the Transformer?

The Transformer operates on a single timestep: it contextualises the 15 sensor readings at time t but has no memory of t−1, t−2, …. Consider the core scenario:

```
Frame 1: oncoming_dist = 0.8   → "oncoming, but far" Frame 5: oncoming_dist = 0.4   →
"oncoming, medium" Frame 10: oncoming_dist = 0.2  → same distance, but DIFFERENT
velocities possible
```

Two scenarios give the same current reading of 0.2 but completely different safe actions: one truck closing at 0.06/frame (must wait longer) vs. one at 0.02/frame (safe to merge now). The LSTM's hidden state encodes the history of distance readings and implicitly learns to estimate relative velocity.

Why LSTM not GRU? Both are valid. LSTM has two state vectors (cell state c and hidden state h), unlike GRU which has only hidden state h, hence LSTM gives slightly more capacity for long-range memory, relevant here because the car may need to 'remember' that an oncoming truck was detected 2–3 seconds ago before deciding to merge. In practice the performance difference is small; LSTM is the SB3-contrib default for RecurrentPPO.

3.3 Why Behavioural Cloning Before PPO?

RL from scratch on a sparse reward signal is sample-inefficient. The agent spends millions of steps randomly wandering before discovering that passing the obstacle is rewarded. Behavioural cloning solves the cold-start problem:

- Stage 1 (BC): 30 epochs on ~50 000 expert state-action pairs. Cross-entropy loss. Trains only the Transformer. Takes minutes. Produces a feature extractor that already understands 'obstacle ahead → steer left'.
- Stage 2 (PPO): 2 000 000 timesteps. The pre-trained Transformer weights are loaded; the LSTM and heads are trained from scratch via PPO. The agent starts from a sensible policy rather than random noise.
-

3.4 Why PPO and Not Other RL Algorithms?

Property	Why it matters for our project
On-policy	No replay buffer needed. Critical for the Pi which has 4 GB shared between OS, Python, and inference.
Clipped surrogate objective	Prevents catastrophic policy updates. The clip_range=0.2 parameter means the policy cannot change more than ±20% per gradient step.
Entropy regularisation	ent_coef=0.05 adds a bonus for policy entropy. This encourages exploration: the agent does not prematurely commit to a single strategy (e.g., always waiting) before discovering that merging left is sometimes better.

Why not SAC (Soft Actor-Critic)? SAC is primarily designed for **continuous action spaces** (e.g., steering angles, throttle values), whereas our environment uses a **discrete set of actions** (5 choices like forward, left, right, etc.).

While discrete variants of SAC do exist, they are less stable and not well supported for recurrent policies (LSTM) in our framework.

Why not DQN? DQN estimates the value **Q(s, a)** for each possible action separately. This means, For 5 actions, the network must effectively evaluate each action at every step. This leads to higher computational cost during inference.

3.5 Why 8 Parallel Environments?

SubprocVecEnv spawns 8 independent environment processes. The benefits are:

- Data diversity: 8 simultaneous episodes with different random seeds mean the agent sees varied obstacle positions, sizes, and oncoming truck speeds within each rollout batch. This reduces overfitting to a single scenario configuration.
- No GIL bottleneck: Python's Global Interpreter Lock prevents true multi-threading for CPU-bound work. SubprocVecEnv uses separate OS processes, bypassing the GIL entirely.

4. Observation Space — The 15-Dimensional State Vector

Index — Name	Meaning and Normalisation
0 — us_left	Left ultrasonic (−35°): raw distance / SENSOR_RANGE. 0.0 = touching, 1.0 = nothing in range.
1 — us_center	Centre ultrasonic (0°). The primary collision-avoidance signal.
2 — us_right	Right ultrasonic (+35°). Right-lane availability indicator.
3 — speed	Current forward speed / MAX_SPEED (4.0 px/step). 0.0 = stopped, 1.0 = full speed.
4 — lateral	us_left / (us_left + us_right + ε). Approximates lateral position in lane. 0.5 = centred.
5 — static_det	Binary: ObstacleTracker detected a stationary obstacle in the path.
6 — static_dist	Normalised distance to nearest static obstacle. 1.0 = none detected.
7 — oncoming_det	Binary: a moving oncoming obstacle is detected by ObstacleTracker.
8 — oncoming_dist	Normalised distance to oncoming obstacle.
9 — oncoming_spd	Oncoming obstacle's estimated relative speed, normalised.
10 — left_free	Binary: left adjacent lane is unobstructed within 200 px look-ahead.
11 — right_free	Binary: right adjacent lane is unobstructed.
12 — traffic_signal	Camera output: 0.0 = red/stop-sign, 1.0 = green. Controls the traffic-light/stop-sign scenario.
13 — passed	Binary: car's y has passed the broken truck's y by > 80 px. Triggers return-to-centre reward.
14 — stop_sign	Camera output: 0.0 = no stop sign, 1.0 = stop sign detected.

4.1 ObstacleTracker — Velocity Estimation Without Optical Flow

The HC-SR04 provides scalar distance, not velocity. The ObstacleTracker maintains a ring buffer of the last 5 centre ultrasonic readings and computes a finite-difference velocity estimate:

```
v_relative = -mean(diff(history))   # change in distance per step v_car =
mean(speed_history) * (MAX_SPEED / SENSOR_RANGE) v_object = v_relative - v_car
# isolate obstacle's own velocity
```

If v_object > 0.005: the obstacle is moving toward the car → classified as oncoming. If |v_object| < 0.005: stationary obstacle. This cheap computation (no camera, no optical flow, no additional hardware) gives the agent the temporal signal it needs

5. Training Pipeline

5.1 Stage 1: Expert Data Generation (datasetbc.py)

A hand-coded ExpertPolicy implements a finite state machine with 6 rules (traffic light, emergency stop, obstacle avoidance, lane selection, return-to-centre, default forward). 500 episodes are simulated with domain randomisation. Only successful episodes (passed_obstacle=True) are kept.

Why only successful episodes? Behavioural cloning on failed trajectories teaches the model to reproduce failures. By filtering to successes only, we guarantee that the distribution of demonstrated behaviours reflects competent driving.

5.2 Stage 2: Behavioural Cloning (behav_clone.py)

Hyperparameter	Value & Rationale
learning_rate	1e-3 with StepLR (γ=0.5, step=10). Start aggressive; halve every 10 epochs to fine-tune.
loss	CrossEntropyLoss. Standard for multi-class classification (5 actions).
gradient clip	1.0. Prevents exploding gradients during early training when weights are random.
output	models/bc/best_bc.pth: model_state_dict + features_state_dict (the Transformer weights that get loaded into PPO).

5.3 Stage 3: Recurrent PPO (ppo_rl.py)

Hyperparameter	Value & Rationale
total_timesteps	2 000 000. (2 million)
n_envs	8.
n_steps	128 per env per rollout. Shorter than vanilla PPO (2048) because BPTT through 128 LSTM steps is already expensive. Longer sequences do not improve performance here.
gamma	0.99. High discount factor: the car must plan the full overtake (≈ 150 steps) before receiving the pass bonus. γ=0.99 means a reward 150 steps ahead is worth 0.99^150 ≈ 0.22 — still meaningful.
gae_lambda	0.95. Generalised Advantage Estimation λ. Balances bias (λ→0, use value baseline only) against variance (λ→1, use raw returns). 0.95 is the standard recommendation (Schulman 2016).
clip_range	0.2. PPO canonical value. Empirically robust across domains.

ent_coef	0.05. Slightly higher than the 0.01 default to encourage exploration in the early phase when the agent discovers lane-change manoeuvres.
learning_rate	1e-4 (constant). Lower than BC because we are fine-tuning pre-trained weights; large LR would destroy the BC initialisation.
lstm_hidden_size	64. Compact: 64 × 4 (gates) × (128+64) weight params per cell = 49 152 params for the LSTM. Fits in 192 KB at FP32.

5.4 Reward Function Design

Reward Term	Value & Design Rationale
Pass bonus	+50.0 (one-time). Fires once when car clears the obstacle. Magnitude chosen to be clearly above the cumulative progress reward for the same distance, so the agent is incentivised to complete the manoeuvre rather than hover near the obstacle.
Goal bonus	+200.0. Fires when car exits the top of the screen. This sparse terminal reward ensures the agent does not dawdle after passing.
Collision	−200.0. Equal in magnitude to the goal bonus: crashing is exactly as bad as succeeding is good. This symmetry prevents the agent from accepting a crash as 'worth it' under any rational discount.
Off-road	−150.0. Slightly less than collision: driving off road is bad but less immediately catastrophic. Avoids the agent treating off-road as equivalent to collision.
Proximity < 0.15	−3.0/step. Continuous penalty for being very close to any obstacle. Teaches the agent to maintain a safety margin rather than brushing past obstacles.
Proximity < 0.30	−1.0/step. Softer warning zone.
Time penalty	−0.05/step. Small constant cost. Without it the agent may learn to stop and wait indefinitely (safe, but non-functional). This creates urgency proportional to time elapsed.
Steering penalty	−0.05 per steer action. Discourages oscillating left-right. Encourages smooth, purposeful steering.
Centre-lane penalty	−2.0 × (lateral_offset / LANE_W) after passing. After the obstacle is passed, penalise lateral distance from centre lane. Teaches the return-to-centre behaviour.
Dangerous merge	−20.0. Extra penalty for STEER_LEFT when oncoming_det=1 and oncoming_dist < 0.4. Teaches 'wait for oncoming to clear' without hard-coding it as a rule.

6. Perception Pipeline

6.1 Traffic Sign Classifier (MobileNetV3-Small)

A fine-tuned MobileNetV3-Small classifies camera frames into three classes: green (go), red (stop), stop_sign (stop). Input resolution is 96×96 . small enough for 30+ FPS on the Pi, large enough for reliable classification.

Why MobileNetV3-Small and not YOLOv8? YOLO performs detection + localisation simultaneously and is overkill for traffic light state classification. We only need to know 'is the light red or green?' is a pure classification task. MobileNetV3-Small has 2.9 M parameters vs. YOLOv8-nano's 3.2 M, and runs faster on single-class classification inference.

Why freeze the first 40 layers? MobileNetV3 pre-trained on ImageNet has already learned edge detectors, texture filters, and shape detectors in its early layers. These are universal and applicable to traffic signs. Fine-tuning only the later layers (which learn class-specific features) prevents catastrophic forgetting of these universal features.

6.2 Static Quantisation of the Classifier (classifier_int8.py)

The FP32 ONNX classifier is quantised to INT8 using ONNX Runtime's static quantisation with QDQ (Quantise-DeQuantise) format and percentile calibration.

Why static quantisation, not dynamic?
Static quantisation pre-computes the activation quantisation parameters (scale and zero-point) from a representative calibration dataset (300 images, 100 per class). Dynamic quantisation only quantises weights, not activations — it computes activation scale on the fly at inference time. For a classification CNN where the activation distributions are stable (the camera always sees the same scale of objects), static quantisation is better over dynamic.

6.3 Dynamic Quantisation of the Policy (ppo_rl_int8.py)

The RL policy uses dynamic quantisation, which is the opposite choice to the classifier. This is intentional.

Why dynamic for the policy, not static?
The RL policy processes a 15-float state vector that changes every timestep as the car moves. The LSTM hidden state also evolves continuously. Unlike the classifier (fixed camera distribution → stable activations), the policy's internal activations vary enormously depending on the driving situation: approaching an obstacle vs. clear road vs. waiting for oncoming traffic. Pre-computing activation scale from a calibration set would capture one distribution; at inference time the actual distribution would be different, causing accuracy loss. Dynamic quantisation measures activation range on the fly, accepting slightly higher per-inference compute in exchange for correct quantisation at every timestep.

7. Deployment on Raspberry Pi

7.1 ONNX Runtime Inference

Both the classifier and the policy are served via ONNX Runtime with CPUExecutionProvider. ONNX Runtime's execution engine applies graph-level optimisations (constant folding, operator fusion, memory planning) that are not available when running PyTorch directly on CPU.

Key session options for the Pi:

- intra_op_num_threads = 4 — matches the Pi 4B's 4 ARM cores.
- inter_op_num_threads = 1 — no parallelism between operators (overhead outweighs benefit for small models).
- graph_optimization_level = ORT_ENABLE_ALL — enable all graph optimisations.

7.2 LSTM State Management at Inference

The ONNX policy model takes three inputs: observation (1, 15), h_in (1, 1, 64), c_in (1, 1, 64), and returns three outputs: action_logits (1, 5), h_out (1, 1, 64), c_out (1, 1, 64). The calling code must:

- Initialise h and c as zeros at the start of each driving session.
- Pass the returned h_out and c_out as h_in and c_in on the next call.
- Reset to zeros if the car crashes or the session ends.

This statefulness is what gives the deployed system temporal memory. The LSTM 'remembers' that an oncoming truck was detected 0.5 seconds ago and its distance was decreasing, even though the current observation vector only shows the present state.

8. Repository Structure

```
autonomous-drive/
├── simulation/
│   ├── __init__.py          # Gymnasium env registration
│   └── environment.py       # DrivingEnv: full simulation
├── training/
│   ├── transformer.py       # TransformerFeatures, TransformerBlock, MHSA
│   ├── behav_clone.py       # Stage 1: BC training
│   ├── datasetbc.py         # Expert data generation
│   └── ppo_rl.py            # Stage 2: RecurrentPPO training
├── perception/
│   ├── mobnetv3.py          # MobileNetV3 classifier training
│   └── classifier_int8.py   # ONNX export + INT8 static quantisation
├── deployment/
│   └── ppo_rl_int8.py       # Policy ONNX export + INT8 dynamic quant
├── models/                  # Saved checkpoints (git-ignored)
│   ├── bc/best_bc.pth
│   ├── classifier/best_classifier.pth
│   └── rl_ppo/final_model.zip
├── data/                    # Datasets (git-ignored)
│   ├── expert/              # states.npy, actions.npy
│   └── traffic_signs/       # ImageFolder: train/, val/
├── onnx/                    # Exported models
│   ├── policy_int8.onnx
│   └── classifier_int8.onnx
├── test.py                  # Manual keyboard control for sim testing
├── requirements.txt
└── README.md
```